

BASİT GİRİŞ ÇIKIŞ İŞLEMLERİ

Modüler Programlama

C# programlama dili, aynı zamanda modüler bir programlama dilidir. Oluşturulan ve önceden test edilip doğrulanmış modüller olan **sınıf kütüphaneleri** (**class library**) yazacağımız kodlara **using** yönergesi ile dahil edilir.

```
using System; // System adındaki isim uzayındaki bileşenleri kullanacağız!.
//System isim uzayının içindeki Console sınıfını kullanabilmek için.
namespace ÖrnekUygulama // ÖrnekUygulama adında bir isim Uzayı
{
    class Program // Programı başlatacak Main yöntemini içeren sınıf
    {
        static void Main(string[] args) // Programın başlangıç noktası
        {
            Console.WriteLine("Merhaba"); // Konsola "Merhaba" yazdırır
        }
    }
}
```

Modüllere ayırmanın **soyutlama** (**abstraction**), **değişim yönetimi** (**change management**) ve **yeniden kullanım** (**reusing**) gibi üstünlükleri vardır. Modüllere ayrılan bir kodun bakımı da daha kolaydır. Ancak daha fazla fonksiyonun çağırılması, işlemciyi yorar. Çünkü her fonksiyon çağırısında işlemcinin mevcut durumu ve kaydediciler belleğe itilir ve fonksiyondan geri döndüğünde eski duruma dönmek için bu veriler tekrar geri çekilir. İşlemciye yüklenen bu **çağırma yükü** (**calling overhead**) günümüz bilgisayarlarında oldukça ihmal edilebilir seviyededir. Ancak milisaniyeler bazında yapılması gereken bir iş varsa böyle zaman kritik işler gerçekleştirmek için bu durum göz önüne alınmalıdır.

C# programlama dilinde en çok kullanacağımız modüllerden biri de giriş çıkış işlemlerinin (**input output-IO**) tanımlı olduğu **System** ve **System.IO** sınıf kütüphanesi ön tanımlı olarak koda dahil edilmiştir. Böylece konsola bir şey yazmak ve konsoldan bir şey okumak için **Console** sınıfını doğrudan kullanırsınız. Bunların yanında **Math**, **Random**, **string** gibi birçok sınıf kullanır hale geliriz^{15,16}.

Konsolda Veri Okuma ve Yazma

Konsoldan veri okuma ve yazma **Console** nesnesi üzerinden yapılır. Nesnenin **Write** yöntemi ile imlecin bulunduğu yerden itibaren parametre olarak girilen metin konsola yazılır. Nesnenin **WriteLine** yöntemi ile imlecin bulunduğu yerden itibaren parametre olarak girilen metin konsola yazılır ve alt satıra inilir. **Write** ve **WriteLine** yönteminin ilk parametresi metin olmak zorunda değildir.

```
// See https://aka.ms/new-console-template for more information
Console.Write("Merhaba ");
Console.WriteLine("İlhan ");
int yaş = 50;
Console.WriteLine(yaş);
float kilo = 99.5f;
Console.WriteLine(kilo);
decimal para = 545.5M;
Console.WriteLine(para);
/*
Merhaba İlhan
50
99,5
545,5
*/
```

¹⁵ <https://learn.microsoft.com/en-us/dotnet/api/system?view=net-9.0>

¹⁶ <https://learn.microsoft.com/en-us/dotnet/api/system.io?view=net-9.0>

Write ve **WriteLine** yönteminin ilk parametresi, biçimlendirme metni olarak kullanılabilir. İzleyen her bir parametrenin sıfırdan başlayan bir numarası vardır.

```
// See https://aka.ms/new-console-template for more information
int sayi1 = 20, sayi2 = 30;
Console.WriteLine("Sayılar:{0}, {1}",sayi1,sayi2);
int toplam = sayi1 + sayi2;
Console.WriteLine("Sayıların Toplamı:{0}", toplam);

/*
Sayılar:20, 30
Sayıların Toplamı:50
*/
```

Dizgiler

Metinleri kod içerisinde işlemek için aslında karakter dizileri olan **dizgiler** (**string**) kullanılır. Ancak, daha yaygın uygulama olarak, bir dizgi değişkeni bildirmek için **string** anahtar sözcüğünü kullanılır. **string** anahtar sözcüğü, **System.String** sınıfı için bir takma addır. Aşağıdaki şekilde tanımlanabilirler;

```
// See https://aka.ms/new-console-template for more information
string adı, soyadı;
adı = "İlhan";
soyadı = "ÖZKAN";

char[] harfler = { 'M', 'e', 'r', 'h', 'a', 'b', 'a' }; //Karakter Dizisi
string[] metindizisi = { "Merhaba", "Dünya" }; //String Dizisi

string tamAdı = adı + " " + soyadı;
Console.WriteLine("Tam Adı: {0}", tamAdı);

string selamlama = new string(harfler);
Console.WriteLine("Selamlama: {0}", selamlama);

string mesaj = String.Join(" ", metindizisi);
Console.WriteLine("Message: {0}", mesaj);

DateTime zaman = new DateTime(2025, 3, 30, 16, 04, 0);
string metinOlarakZaman = String.Format("{0:t} on {0:D}", zaman);
Console.WriteLine("Zaman: {0}", metinOlarakZaman);

/*Program Çıktısı:
Tam Adı: İlhan ÖZKAN
Selamlama: Merhaba
Message: Merhaba Dünya
Zaman: 16:04 on 30 Mart 2025 Pazar
*/
```

Dizgilerin karakter uzunluğunu aşağıdaki gibi öğrenebiliriz;

```
// See https://aka.ms/new-console-template for more information
string str = "İlhan Özkan";
int length = str.Length;
```

Temel Dizgi İşlemleri

Dizgiler toplama işlemi ile birleştirilebilir;

```
// See https://aka.ms/new-console-template for more information
string s1 = "İlhan";
string s2 = "Özkan";
string s3 = s1 + " " + s2; //"İlhan Özkan"
```

```
string s4 = string.Concat(s1, " ", s2); //"İlhan Özkan"
string s5 = string.Join(" ", s1, s2); //"İlhan Özkan"
string s6 = string.Format("{0} {1}", s1, s2); //"İlhan Özkan"
string s7 = string.Concat(s1, " ", s2); //"İlhan Özkan"
string s8 = string.Join(" ", new string[] { s1, s2 }); //"İlhan Özkan"

string s9 = $"{s1} {s2}"; //"İlhan Özkan"
```

String.Format() yöntemini kullanarak dizgideki bir veya daha fazla öğeyi bir dizgide birleştirebiliriz;

```
String.Format("Yaşım: {0} ve Adım: {1}", 45, "İlhan") //Yaşım:45 ve Adım:İlhan
```

Burada ilk argüman biçimlendirme metnidir. Biçimlendirme metni içerisinde sıfırdan başlayarak izleyen argüman numaraları tırnaklı parantez içerisinde verilir.

Dizgiler içerisinde UTF-8 karakterleri 4 karakterli onaltılık sayı olarak da verilebilir;

```
string s = "\u0130lhan"; //"İlhan"
```

Dizgilere ilk değer çift tırnak karakterleri içerisinde verilir. Peki metin çift tırnak karakteri içeriyorsa ne yapılır? İşte burada çift tırnak karakteri gibi bazı karakterler **kaçış dizisi** (**escape sequence**) olarak adlandırılır;

Escape sequence	Karakter Adı	Unicode Kodu
\'	Tek Tırnak	0x0027
\"	Çift Tırnak	0x0022
\\	Ters Bölü	0x005C
\0	Boş (Null)	0x0000
\a	Uyarı Sesi (Alert)	0x0007
\b	Geri (Backspace)	0x0008
\e	Esc (Escape)	0x001B
\f	Yeni Sayfa (Form feed)	0x000C
\n	Yeni Satır(New line)	0x000A
\r	Satır Başı (Carriage return)	0x000D
\t	Yatay Sekme (Horizontal tab)	0x0009
\v	Dikey Sekme(Vertical tab)	0x000B
\u	Unicode escape sequence (UTF-16)	\uHHHH (0000 - FFFF) \u00E7 = "ç"
\U	Unicode escape sequence (UTF-32)	\U00HHHHHH (000000-10FFFF) \U0001F47D = "🐶"
\x	Unicode escape sequence "\u" ya benzer fakat onaltılık rakam sayısı değişir.	\xH[H][H][H] (0 - FFFF) \x00E7 = \x0E7 = \xE7 = "ç"

Tablo 17. Kaçış Dizisi Karakterleri

Metni belirlenen bir aralıkta sağa ya da sola **yaslama** (**padding**) da yapılabilir;

```
string s = "\u0130lhan \ '61\ '";
Console.WriteLine("\u0130lhan \ '61\ '"); //İlhan '61'
string paddedLeft = s.PadLeft(10, ' '); //Sola boşluk ekle
Console.WriteLine(paddedLeft); // İlhan
string paddedRight = s.PadRight(10, '+'); //Sağa + karakteri ekle
Console.WriteLine(paddedRight); //İlhan+++++
```

Bit tamsayıyı da metin olarak herhangi bir tabana çevirebiliriz;

```
Int32 Number = 252;
Console.WriteLine(Convert.ToString(Number, 2)); //11111000
Console.WriteLine(Convert.ToString(Number, 8)); //370
Console.WriteLine(Convert.ToString(Number, 10)); //252
Console.WriteLine(Convert.ToString(Number, 16)); //FC
```

Değer tipler `ToString()` yöntemi ile metne çevrilebilir;

```
int intValue = 31;
string zeroPaddedInteger = intValue.ToString("000"); //031
string customFormat = intValue.ToString("Değer 0 dır."); //Değer 31 dır.

double doubleValue = 10.456;
string roundedDouble = doubleValue.ToString("0.00"); //10,46
string integerPart = doubleValue.ToString("00"); //10
string customDoubleFormat = doubleValue.ToString("Değer 0.0\'dır."); //Değer 10.5 dır.

DateTime currentDate = DateTime.Now; // 14/05/2025 10:38:03
string dateTimeString = currentDate.ToString("dd-MM-yyyy HH:mm:ss"); // "14-05-2025 10:38:03"
string dateOnlyString = currentDate.ToString("dd-MM-yyyy"); // "14-05-2025"
string dateWithMonthInWords = currentDate.ToString("dd-MMMM-yyyy HH:mm:ss");
// "14-Mayıs-2025 10:38:03"
```

Tam Dizgiler

`Write` ve `WriteLine` yönteminin ilk parametresi, **tam dizgi** (**verbatim string**) olabilir. Bu tam metin, biçimlendirme amacıyla kullanılabilir.

```
// See https://aka.ms/new-console-template for more information
string filename1 = @"c:\documents\files\u0066.txt"; // Tam metin (verbatim string)
string filename2 = "c:\\documents\\files\\u0066.txt"; // Sıratan metin (string)
Console.WriteLine(filename1);
Console.WriteLine(filename2);
Console.WriteLine(@"İlhan
                  Özkan  " \  *");
/*
c:\documents\files\u0066.txt
c:\documents\files\u0066.txt
İlhan
                  Özkan  " \  *
*/
```

Dizgi Enterpolasyonu

`Write` ve `WriteLine` yönteminin ilk parametresi, değişken kimliklerini içerebilen bir metin (**string interpolation**) olabilir. Bu durumda değişmez metninin önüne dolar karakteri konur. Dolayısıyla izleyen parametreleri kullanmak zorunda kalmayız. Basit aritmetik ifadeleri de yazabiliriz.

```
// See https://aka.ms/new-console-template for more information
string isim= "İlhan";
int yaş = 50;
float kilo = 99.5f;
decimal para = 545.5M;
Console.WriteLine($"Adınız: {isim}, Kilonuz: {kilo}, Paranız: {para}");
DateTime date = DateTime.Now;
Console.WriteLine($"Merhaba, Bugün {date.Date} {date.DayOfWeek}");
/*
Adınız: İlhan, Kilonuz: 99,5, Paranız: 545,5
Merhaba, Bugün 21.02.2025 00:00:00 Friday
*/
var StrWithMathExpression = $"1 + 2 = {1 + 2}"; // "1 + 2 = 3"
```

Konsola Yazılan Metni Biçimlendirme

Tarih Biçimlendirme

```
// See https://aka.ms/new-console-template for more information
DateTime date = new DateTime(1971, 09, 01, 11, 53, 24);
```

```

Console.WriteLine(String.Format("{0:dd}", date)); //Yalnızca Günü Yazar:01
//Yukarıdaki Kodun Kısa Hali:
Console.WriteLine($"{date:dd}");
Console.WriteLine("{0:dd}",date);

Console.WriteLine(String.Format("{0:dd/MM/yyyy}", date));
//Günü Ayı ve Yılı Yazar: 01.09.1971
//Yukarıdaki Kodun Kısa Hali:
Console.WriteLine($"{date:dd/MM/yyyy}");
Console.WriteLine("{0:dd/MM/yyyy}",date);

Console.WriteLine(String.Format("{0:dd/MM/yyyy HH:mm:ss}", date));
//Günü Ayı Yılı Saat Dakika ve Saniyeyi Yazar: 01.09.1971 11:53:24
//Yukarıdaki Kodun Kısa Hali:
Console.WriteLine($"{date:dd/MM/yyyy HH:mm:ss}");
Console.WriteLine("{0:dd/MM/yyyy HH:mm:ss}",date);

//Günü İsim Olarak Almanca Yazmak için:
Console.WriteLine(String.Format(new System.Globalization.CultureInfo("de-DE"),
                                "{0:dddd}",
                                date)); //Mittwoch

```

Tarih ve Zamanı aşağıdaki şekilde biçimlendirebiliriz;

```

// See https://aka.ms/new-console-template for more information
DateTime date = new DateTime(1971, 09, 01, 11, 53, 24);
Console.WriteLine($"{date:d}"); //Günü Yazar: 1.09.1971
Console.WriteLine($"{date:dd}"); //Günü Yazar: 01
Console.WriteLine($"{date:ddd}"); //Kısa Günü Yazar: Çar
Console.WriteLine($"{date:dddd}"); //Uzun Günü Yazar: Çarşamba
Console.WriteLine($"{date:D}"); //Uzun Tarih Yazar: 1 Eylül 1971 Çarşamba
Console.WriteLine($"{date:f}"); //Uzun Tarih ve Kısa Saati Yazar: 1 Eylül 1971 Çarşamba 11:53
Console.WriteLine($"{date:F}"); /* Uzun Tarih ve Uzun Saati Yazar:
                                1 Eylül 1971 Çarşamba 11:53:24 */
Console.WriteLine($"{date:g}"); //Kısa Tarih ve Kısa Saati Yazar: 1.09.1971 11:53
Console.WriteLine($"{date:hh}"); //Saatin 12 Saatlik Formatı: 11
Console.WriteLine($"{date:mm}"); //Dakika Yazar: 53
Console.WriteLine($"{date:MM}"); //Ayı Sayısal Olarak Yazar: 09
Console.WriteLine($"{date:MMM}"); //Ayı Kısa Olarak Yazar: Eyl
Console.WriteLine($"{date:MMMM}"); //Ayı Uzun Olarak Yazar: Eylül
Console.WriteLine($"{date:ss}"); //Saniye Yazar: 24
Console.WriteLine($"{date:r}"); //RFC1123 Formatında Yazar: Wed, 01 Sep 1971 11:53:24 GMT
Console.WriteLine($"{date:t}"); //Kısa Saati Yazar: 11:53
Console.WriteLine($"{date:T}"); //Uzun Saati Yazar: 11:53:24
Console.WriteLine($"{date:tt}"); //AM/PM Formatında Yazar: ÖÖ
Console.WriteLine($"{date:y}"); //Ay ve Yılı Yazar: Eylül 1971
Console.WriteLine($"{date:yy}"); //Yılı Kısa Olarak Yazar: 71
Console.WriteLine($"{date:yyyy}"); //Yılı Uzun Olarak Yazar: 1971

```

Para Birimi Biçimlendirme

Para birimi biçimlendiricisi olarak "c" biçim belirleyicisi kullanılır. Bir sayıyı para birimi miktarını temsil eden bir **dizgiye** (string) dönüştürür;

```

// See https://aka.ms/new-console-template for more information
using System.Globalization;
double para = 2510.752525;

//Sistem tarafından ön tanımlı olarak bozukluk iki hane yazılır.
//Programın çalıştığı bilgisayar türkçe ayarlarına sahip:
Console.WriteLine(string.Format("{0:c}", para)); // ?2.510,75
Console.WriteLine($"{para:C}"); // ?2.510,75

```

```
//Bozukluk hane sayısı aşağıdaki şekild belirtilir:
Console.WriteLine(string.Format("{0:c1}", para)); // ?2.510,8
Console.WriteLine(string.Format("{0:c2}", para)); // ?2.510,75
Console.WriteLine(string.Format("{0:c3}", para)); // ?2.510,753
Console.WriteLine(string.Format("{0:c4}", para)); // ?2.510,7525
Console.WriteLine();

NumberFormatInfo nfi = new CultureInfo("en-US", false).NumberFormat;
nfi.CurrencySymbol = "$";
nfi.NumberGroupSeparator = ",";
nfi.NumberDecimalSeparator = ".";
nfi.CurrencyPositivePattern = 0;
Console.WriteLine(string.Format(nfi, "{0:C}", para)); // $2,510.75 - default
nfi.CurrencyPositivePattern = 1;
Console.WriteLine(string.Format(nfi, "{0:C}", para)); // 2,510.75$
nfi.CurrencyPositivePattern = 2;
Console.WriteLine(string.Format(nfi, "{0:C}", para)); // $ 2,510.75
nfi.CurrencyPositivePattern = 3;
Console.WriteLine(string.Format(nfi, "{0:C}", para)); // 2,510.75 $
```

Sayı Biçimini Değiştirme

`NumberFormatInfo` hem tam sayı hem de gerçek sayıları biçimlendirmek için kullanılabilir.

```
// See https://aka.ms/new-console-template for more information
using System.Globalization;
double number = 1234567.89;

var invariantResult = string.Format(CultureInfo.InvariantCulture, "{0:#,###,##}", number);
Console.WriteLine(invariantResult); // 1,234,568
var customProvider = new NumberFormatInfo
{
    NumberDecimalSeparator = "_Ondalık_",
    NumberGroupSeparator = "_Bindelik_",
};

var customResult = string.Format(customProvider, "{0:#,###.##}", number);
Console.WriteLine(customResult); // 1_Bindelik_234_Bindelik_567_Ondalık_89
```

Sağa sola yaslama;

```
// See https://aka.ms/new-console-template for more information
int sayı = 12;
string metin = "alo";
string sol=string.Format("SOL: string: |{0,-5}| int:|{1,-5}|", metin, sayı);
Console.WriteLine(sol); //SOL: string: |alo | int:|12 |
string sağ=string.Format("SAĞ: string: |{0,5}| int:|{1,5}|", metin, sayı);
Console.WriteLine(sağ); //SAĞ: string: | alo| int:| 12|
```

Sayı sistemleri;

```
// See https://aka.ms/new-console-template for more information
int sayı = 1234;

string hexstring=string.Format("Hexadecimal: {0:x2}-{0:X4}", sayı); // Hexadecimal: 4d2-04D2
Console.WriteLine(hexstring);
Console.WriteLine($"{{sayı:x2}}-{{sayı:x4}}"); //4d2-04D2

// Tamsayılarda bindelik ayırıcı:
string bindlikstring=string.Format("Bindelik Ayırıcı: {0:#,#}", sayı);
Console.WriteLine(bindlikstring); //Bindelik Ayırıcı: 1.234
Console.WriteLine($"{{sayı:#,#}} |{{sayı,10:#,#}}|"); //1.234 | 1.234|

decimal ondalık = 1234.5678m;
```

```
string ondalıkstr=string.Format("Ondalık: {0:0.000}-Yüzde: {0:0.00%}",ondalık);  
Console.WriteLine(ondalıkstr); //Ondalık: 1234.568-Yüzde: 123456.78%  
Console.WriteLine($"{ondalık:0.000} |{ondalık,10:0.000%}|"); //1234.568 | 1234.568%|
```

Metinden Değer Tiplere Dönüşüm

`ReadLine` yöntemi ile konsoldan bir satır metin okunabilir. Sayı okumamız durumunda bu metnin istenilen değişken tipine dönüştürülmesi gerekir.

```
// See https://aka.ms/new-console-template for more information  
double sayi;  
Console.WriteLine("Karekökü Alınacak Sayıyı Giriniz");  
string okunan = Console.ReadLine();  
sayi = Convert.ToDouble(okunan);  
Console.WriteLine("Hesaplanan Karekök: {0}",Math.Sqrt(sayi));  
string str = "6161";  
int num = int.Parse(str);  
Console.WriteLine(num);  
string str2 = "6100";  
if (int.TryParse(str2, out int result)) {  
    Console.WriteLine(result);  
} else {  
    Console.WriteLine("Dönüştürme Başarısız Oldu.");  
}  
/*  
Adınız: İlhan, Kilonuz: 99,5, Paranız: 545,5  
Merhaba, Bugün 21.02.2025 00:00:00 Friday  
6161  
6100  
*/
```